# ParaThink: Speech Frontend, Parallel Thinking and Expression

Continuous observation / Shared context / One playback owner

Incoming user audio

## Speech Frontend: SoulX-Duplug + SenseVoiceSmall

**Online ASR + Text Assembly**
SenseVoiceSmall (first version)
Versioned transcript updates

Transcript

**SoulX-Duplug User-State Model**
Audio encoding + transcript input
Five native user states

Future CoT input

Continuous audio observation; publish text before user_complete. Nemotron: later evaluation.

Versioned transcript

Five user states

## Shared Context: Available Dialogue + Runtime State

Versioned transcript + generated history / SoulX-Duplug user states / Layer 2 CoT
Backchannel timing suggestions / Layer 1 output / Listen–Speak state
Interruption events / generated, submitted, canceled and played records

Read / Timing

**Layer 0**
**Backchannel Timing**
System backchannel
Timing prediction

Read / Output

**Layer 1**
**Expression**
Read all five user states
Reply after user_complete
Request thinking

Request

Read / CoT

**Layer 2**
**Async Thinking**
Publish CoT only
No turn-action advice
Update one active task

Frontend observation, expression and thinking can run concurrently.

System backchannel

Speech text

## Text FIFO

Submitted speech text / system backchannel, in order
Chunks carry output_id + chunk_id; accept active output only

Consume in order

Valid user_nonidle
During active output: stop
Bypass FIFO / no LLM wait

## Output Runtime: Streaming TTS + Player

Single arbitration point / single owner of audible output
Priority stop: halt audio + cancel pending output
Reject late text / audio carrying an invalid output_id

Actual Listen / Speak state + interruption events

Audio heard by the user: never revoked

### On a valid stop: execution in the same output runtime

1. **Close old output_id**
Block stale submissions

2. **Halt + cancel**
Cancel queued / buffered output
Discard late text / audio

3. **Closure acknowledged**
Publish Listen + interruption
Keep generated history

4. **New Layer 1 reply**
Read matching user_complete
Fresh output_id; new text

Ordinary replies begin after Layer 1 reads user_complete in the current turn; no separate reply call.

Target architecture; integration pending. Dotted feedback: future CoT adaptation and training, not implemented.